

ЛАБОРАТОРНА РОБОТА №3

Структурні шаблони

Мета: навчитися реалізовувати структурні шаблони проєктування *Адаптер*, *Декоратор*, *Міст*, *Компонувальник*, *Проксі*, *Легковаговик*.

Хід роботи:

Завдання 1: Адаптер

1. Створіть клас *Logger*, який буде мати методи *Log()*, *Error()*, *Warn()*, які виводять повідомлення в консоль різними кольорами (зеленим, червоним і оранжевим відповідно).
2. Створіть клас *FileWriter* з методами *Write()*, *WriteLine()*.
3. За допомогою шаблону Адаптер створіть файловий логер.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми

Створюємо клас *Logger*, який виводить текстові повідомлення безпосередньо в консоль. Залежно від методу, текст фарбується у свій колір: зелений для звичайного логу, червоний для помилки і оранжевий для попереджень. Далі створюємо окремий клас *FileWriter*, який вміє працювати з файлами за допомогою стандартних методів *Write* та *WriteLine*. Оскільки інтерфейси цих класів відрізняються, створюємо клас-адаптер *FileLoggerAdapter*. Він реалізує той самий інтерфейс, що й консольний логер, але всередині перенаправляє всі виклики у файл через об'єкт

					ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3						
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи				Літ.	Арк.	Аркушів
Розроб.		Мазурова І. В.									
Перевір.		Левківський В. Л.								1	22
Керівник									ФІКТ Гр. ВТк-25-1		
Н. контр.											
Зав. каф.											

FileWriter.

Лістинг Program.cs:

```
namespace Adapter
{
    public interface ILogger
    {
        void Log(string message);
        void Error(string message);
        void Warn(string message);
    }

    public class Logger : ILogger
    {
        public void Log(string message)
        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine($"[LOG]: {message}");
            Console.ResetColor();
        }

        public void Error(string message)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[ERROR]: {message}");
            Console.ResetColor();
        }

        public void Warn(string message)
        {
            Console.ForegroundColor = ConsoleColor.DarkYellow;
            Console.WriteLine($"[WARN]: {message}");
            Console.ResetColor();
        }
    }

    public class FileWriter
    {
        private string _filePath;

        public FileWriter(string filePath)
        {
            _filePath = filePath;
        }

        public void Write(string text)
        {
            File.AppendAllText(_filePath, text);
        }
    }
}
```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }

    public void WriteLine(string text)
    {
        File.AppendAllText(_filePath, text + Environment.NewLine);
    }
}

public class FileLoggerAdapter : ILogger
{
    private readonly FileWriter _fileWriter;

    public FileLoggerAdapter(FileWriter fileWriter)
    {
        _fileWriter = fileWriter;
    }

    public void Log(string message)
    {
        _fileWriter.WriteLine($"[LOG]: {message}");
    }

    public void Error(string message)
    {
        _fileWriter.WriteLine($"[ERROR]: {message}");
    }

    public void Warn(string message)
    {
        _fileWriter.WriteLine($"[WARN]: {message}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        ILogger consoleLogger = new Logger();
        consoleLogger.Log("Програма працює стабільно.");
        consoleLogger.Warn("Виявлено незначне попередження.");
        consoleLogger.Error("Сталася критична помилка системи.");

        string path = "log.txt";
        if (File.Exists(path)) File.Delete(path);

        FileWriter writer = new FileWriter(path);
        ILogger fileLogger = new FileLoggerAdapter(writer);
    }
}

```

```

fileLogger.Log("Користувач успішно авторизувався.");
fileLogger.Warn("Спроба завантаження великого файлу.");
fileLogger.Error("Не вдалося зберегти зміни в базі.");

Console.WriteLine("\nВміст створеного файлу log.txt:");
Console.WriteLine(File.ReadAllText(path));

Console.ReadKey();
    }
}
}

```

Результат:

```

[LOG]: Програма працює стабільно.
[WARN]: Виявлено незначне попередження.
[ERROR]: Сталася критична помилка системи.

Вміст створеного файлу log.txt:
[LOG]: Користувач успішно авторизувався.
[WARN]: Спроба завантаження великого файлу.
[ERROR]: Не вдалося зберегти зміни в базі.

```

Рис. 1.1. Результат роботи шаблону Адаптер

Завдання 2: Декоратор

1. Ви розробляєте РПГ гру. Створіть класи героїв Warrior, Mage, Palladin.
2. Для героїв створіть інвентар (одяг, зброю, артефакти), який може підходити будь-якому типу героїв, у вигляді декораторів.
3. Важливою вимогою є можливість використання декількох екземплярів інвентаря на герої одночасно.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Реалізовуємо шаблон проєктування Декоратор для створення інвентаря в РПГ грі. В основі архітектури лежить загальний інтерфейс *IHero* та три базові класи героїв: воїн, маг і паладин. Для динамічного додавання спорядження створюємо

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

абстрактний клас *InventoryDecorator*, від якого створено конкретні класи предметів: зброю, броню, плащ, амулет та магична каблучка. Оскільки всі декоратори працюють через єдиний інтерфейс, створені речі є універсальними і підходять будь-якому типу персонажа. Така структура повністю забезпечує виконання технічної вимоги щодо одночасного та повторного використання кількох предметів на одному герої.

Лістинг Program.cs:

```
namespace Decorator
{
    public interface IHero
    {
        string GetDescription();
        int GetPower();
    }

    public class Warrior : IHero
    {
        public string GetDescription() => "Воїн:";
        public int GetPower() => 100;
    }

    public class Mage : IHero
    {
        public string GetDescription() => "Маг:";
        public int GetPower() => 80;
    }

    public class Palladin : IHero
    {
        public string GetDescription() => "Паладин:";
        public int GetPower() => 95;
    }

    public abstract class InventoryDecorator : IHero
    {
        protected IHero _hero;

        public InventoryDecorator(IHero hero)
        {
            _hero = hero;
        }

        public virtual string GetDescription() => _hero.GetDescription();
        public virtual int GetPower() => _hero.GetPower();
    }
}
```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }

    public class WeaponDecorator : InventoryDecorator
    {
        public WeaponDecorator(IHero hero) : base(hero) {}
        public override string GetDescription() => _hero.GetDescription() + "
меч;";
        public override int GetPower() => _hero.GetPower() + 40;
    }

    public class ArmorDecorator : InventoryDecorator
    {
        public ArmorDecorator(IHero hero) : base(hero) {}
        public override string GetDescription() => _hero.GetDescription() + "
броня;";
        public override int GetPower() => _hero.GetPower() + 30;
    }

    public class CloakDecorator : InventoryDecorator
    {
        public CloakDecorator(IHero hero) : base(hero) {}
        public override string GetDescription() => _hero.GetDescription() + "
плащ;";
        public override int GetPower() => _hero.GetPower() + 15;
    }

    public class ArtifactDecorator : InventoryDecorator
    {
        public ArtifactDecorator(IHero hero) : base(hero) {}
        public override string GetDescription() => _hero.GetDescription() + "
амулет;";
        public override int GetPower() => _hero.GetPower() + 20;
    }

    public class RingDecorator : InventoryDecorator
    {
        public RingDecorator(IHero hero) : base(hero) {}
        public override string GetDescription() => _hero.GetDescription() + "
магічна каблучка;";
        public override int GetPower() => _hero.GetPower() + 25;
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            IHero warrior = new Warrior();
            warrior = new WeaponDecorator(warrior);

```

```

        warrior = new ArmorDecorator(warrior);
        warrior = new WeaponDecorator(warrior);
        Console.WriteLine($"Персонаж - {warrior.GetDescription()} | Сила:
{warrior.GetPower()}");

        IHero mage = new Mage();
        mage = new CloakDecorator(mage);
        mage = new ArtifactDecorator(mage);
        mage = new ArtifactDecorator(mage);
        Console.WriteLine($"Персонаж - {mage.GetDescription()} | Сила:
{mage.GetPower()}");

        IHero palladin = new Palladin();
        palladin = new ArmorDecorator(palladin);
        palladin = new RingDecorator(palladin);
        palladin = new WeaponDecorator(palladin);
        Console.WriteLine($"Персонаж - {palladin.GetDescription()} | Сила:
{palladin.GetPower()}");

        Console.ReadKey();
    }
}
}

```

Результат:

```

Персонаж - Воїн: меч; броня; меч; | Сила: 210
Персонаж - Маг: плащ; амулет; амулет; | Сила: 135
Персонаж - Паладин: броня; магічна каблучка; меч; | Сила: 190

```

Рис. 2.1. Робота шаблону Декоратор

Завдання 3: Міст

1. Ви працюєте над графічним редактором. Створіть базовий клас *Shape*.
2. Створіть дочірні до *Shape* класи, *Circle*, *Square*, *Triangle*.
3. За допомогою шаблону Міст додайте можливість рендерингу кожної з фігур як векторної або растрової графіки (вивівши відповідне повідомлення у консоль, наприклад "Drawing Triangle as pixels").

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Створюємо базовий клас *Shape* та його дочірні класи *Circle*, *Square* і *Triangle*. За допомогою шаблону Міст додано можливість рендерингу кожної з фігур як векторної або растрової графіки. При запуску програма виводить у консоль відповідні повідомлення про формат малювання.

Лістинг *Program.cs*:

```
namespace Bridge
{
    public interface IRenderer
    {
        void RenderCircle();
        void RenderSquare();
        void RenderTriangle();
    }

    public class VectorRenderer : IRenderer
    {
        public void RenderCircle() => Console.WriteLine("Drawing Circle as vector lines");
        public void RenderSquare() => Console.WriteLine("Drawing Square as vector lines");
        public void RenderTriangle() => Console.WriteLine("Drawing Triangle as vector lines");
    }

    public class RasterRenderer : IRenderer
    {
        public void RenderCircle() => Console.WriteLine("Drawing Circle as pixels");
        public void RenderSquare() => Console.WriteLine("Drawing Square as pixels");
        public void RenderTriangle() => Console.WriteLine("Drawing Triangle as pixels");
    }

    public abstract class Shape
    {
        protected IRenderer _renderer;

        protected Shape(IRenderer renderer)
        {

```

```

        _renderer = renderer;
    }

    public abstract void Draw();
}

public class Circle : Shape
{
    public Circle(IRenderer renderer) : base(renderer) {}
    public override void Draw() => _renderer.RenderCircle();
}

public class Square : Shape
{
    public Square(IRenderer renderer) : base(renderer) {}
    public override void Draw() => _renderer.RenderSquare();
}

public class Triangle : Shape
{
    public Triangle(IRenderer renderer) : base(renderer) {}
    public override void Draw() => _renderer.RenderTriangle();
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        IRenderer vector = new VectorRenderer();
        IRenderer raster = new RasterRenderer();

        Shape vectorCircle = new Circle(vector);
        Shape rasterSquare = new Square(raster);
        Shape rasterTriangle = new Triangle(raster);

        vectorCircle.Draw();
        rasterSquare.Draw();
        rasterTriangle.Draw();

        Console.ReadKey();
    }
}

```

Результат:

```
Drawing Circle as vector lines
Drawing Square as pixels
Drawing Triangle as pixels
```

Рис. 3.1. Робота шаблону Міст

Завдання 4: Проксі

1. Створіть клас *SmartTextReader*, який вміє читати вміст текстового файлу і перетворювати його на двовірний масив якому зовнішній масив відповідає рядкам тексту, а вкладені масиви відповідають символам у відповідному рядку.
2. Створіть проксі для *SmartTextReader* з логуванням *SmartTextChecker*, який буде виводити інформацію про успішне відкриття, прочитання і закриття файлу, а також буде виводити загальну кількість рядків і символів у прочитаному тексті.
3. Створіть проксі для *SmartTextReader* з обмеженням доступу до певних файлів *SmartTextReaderLocker*. Цей клас в конструкторі приймає регулярний вираз, по якому лімітується доступ до певної групи файлів. Якщо клієнт викликати метод для прочитання такого лімітованого файлу, замість прочитання файлу в консоль має виводитися повідомлення “Access denied!”.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Створюємо клас *SmartTextReader* для читання текстових файлів та їх перетворення у двовірний масив символів. За допомогою шаблону Проксі реалізовано два класи-замісники: *SmartTextChecker* для логування дій та підрахунку рядків і символів , а також *SmartTextReaderLocker* для обмеження доступу до файлів за

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

регулярним виразом. Якщо доступ до файлу лімітовано, програма виводить повідомлення "Access denied!".

Лістинг Program.cs:

```
namespace Proxy
{
    public interface ISmartTextReader
    {
        char[][] ReadTextFile(string filePath);
    }

    public class SmartTextReader : ISmartTextReader
    {
        public char[][] ReadTextFile(string filePath)
        {
            if (!File.Exists(filePath)) return new char[0][];
            string[] lines = File.ReadAllLines(filePath);
            char[][] result = new char[lines.Length][];
            for (int i = 0; i < lines.Length; i++)
            {
                result[i] = lines[i].ToCharArray();
            }
            return result;
        }
    }

    public class SmartTextChecker : ISmartTextReader
    {
        private readonly ISmartTextReader _reader;

        public SmartTextChecker(ISmartTextReader reader)
        {
            _reader = reader;
        }

        public char[][] ReadTextFile(string filePath)
        {
            Console.WriteLine($"[Лог] Успішне відкриття файлу: {filePath}");
            char[][] result = _reader.ReadTextFile(filePath);
            Console.WriteLine($"[Лог] Файл {filePath} успішно прочитано та за-
крито.");

            if (result != null)
            {
                int totalChars = 0;
                foreach (var row in result) totalChars += row.Length;
                Console.WriteLine($"[Статистика] Загальна кількість рядків:
{result.Length}, символів: {totalChars}");
            }
        }
    }
}
```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
    return result;
}

}

public class SmartTextReaderLocker : ISmartTextReader
{
    private readonly ISmartTextReader _reader;
    private readonly Regex _regex;

    public SmartTextReaderLocker(ISmartTextReader reader, string pattern)
    {
        _reader = reader;
        _regex = new Regex(pattern);
    }

    public char[][] ReadTextFile(string filePath)
    {
        if (_regex.IsMatch(filePath))
        {
            Console.WriteLine("Access denied!");
            return null;
        }
        return _reader.ReadTextFile(filePath);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        string regularFile = "public_data.txt";
        string secretFile = "secret_password.txt";

        File.WriteAllText(regularFile, "Hello World\nLine two text.");
        File.WriteAllText(secretFile, "Top secret content");

        ISmartTextReader baseReader = new SmartTextReader();
        ISmartTextReader checkerProxy = new SmartTextChecker(baseReader);
        ISmartTextReader lockerProxy = new
SmartTextReaderLocker(checkerProxy, @"secret.*\.txt");

        Console.WriteLine("--- Спроба читання відкритого файлу ---");
        lockerProxy.ReadTextFile(regularFile);

        Console.WriteLine("\n--- Спроба читання захищеного файлу ---");
        lockerProxy.ReadTextFile(secretFile);
    }
}

```

```

        Console.ReadKey();
    }
}

```

Результат:

```

--- Спроба читання відкритого файлу ---
[Лог] Успішне відкриття файлу: public_data.txt
[Лог] Файл public_data.txt успішно прочитано та закрито.
[Статистика] Загальна кількість рядків: 2, символів: 25

--- Спроба читання захищеного файлу ---
Access denied!

```

Рис. 4.1. Робота шаблону Проксі

Завдання 5: Компонувальник

1. Вам потрібно створити власну мову розмітки *LightHTML*.
2. Кожен елемент розмітки має наслідувати клас *LightNode*.
3. Створіть два дочірніх класи від *LightNode*: *LightElementNode*, *LightTextNode*.
4. *LightTextNode* може містити лише текст.
5. *LightElementNode* може містити будь-які *LightNode*. *LightElementNode* повинен мати інформацію про назву тега, його тип відображення (блочний чи рядковий), тип закриття (одиначний тег, як `<img/>` чи з закриваючим тегом) список CSS класів, кількість дочірніх елементів, а також має бути можливість виводити на екран його `outerHTML` і `innerHTML`.
6. За допомогою своєї мови розмітки виведіть в консоль елемент сторінки на Ваш вибір (наприклад якусь таблицю, список тощо).
7. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Створюємо власну мову розмітки *LightHTML* на основі базового класу *LightNode* та його дочірніх класів *LightElementNode* і *LightTextNode*. За допомогою шаблону Компонувальник реалізовано деревоподібну структуру, де елементи

можуть містити текст або інші вузли, зберігаючи назву тега, типи відображення, закриття та CSS класи. Програма дозволяє автоматично підраховувати кількість дочірніх елементів та виводити властивості *innerHTML* і *outerHTML* для побудованого вузла сторінки.

Лістинг Program.cs:

```
namespace Composite
{
    public abstract class LightNode
    {
        public abstract string InnerHTML { get; }
        public abstract string OuterHTML { get; }
    }

    public class LightTextNode : LightNode
    {
        private readonly string _text;

        public LightTextNode(string text)
        {
            _text = text;
        }

        public override string InnerHTML => _text;
        public override string OuterHTML => _text;
    }

    public class LightElementNode : LightNode
    {
        public string TagName { get; set; }
        public string DisplayType { get; set; }
        public string ClosureType { get; set; }
        public List<string> CssClasses { get; set; } = new List<string>();
        public List<LightNode> Children { get; set; } = new List<LightNode>();

        public int ChildrenCount => Children.Count;

        public LightElementNode(string tagName, string displayType, string closureType)
        {
            TagName = tagName;
            DisplayType = displayType;
            ClosureType = closureType;
        }

        public override string InnerHTML
        {
```

```

        get
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in Children)
            {
                sb.Append(child.OuterHTML);
            }
            return sb.ToString();
        }
    }

    public override string OuterHTML
    {
        get
        {
            string classes = CssClasses.Count > 0 ? $"
class=\"{string.Join(" ", CssClasses)}\" : "";
            if (ClosureType == "single")
            {
                return $"<{TagName}{classes}/>";
            }
            return $"<{TagName}{classes}>{InnerHTML}</{TagName}>";
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;

        LightElementNode list = new LightElementNode("ul", "block",
"paired");
        list.CssClasses.Add("menu-list");

        LightElementNode item1 = new LightElementNode("li", "block",
"paired");
        item1.CssClasses.Add("menu-item");
        item1.Children.Add(new LightTextNode("Головна"));

        LightElementNode item2 = new LightElementNode("li", "block",
"paired");
        item2.CssClasses.Add("menu-item");
        item2.Children.Add(new LightTextNode("Про нас"));

        LightElementNode image = new LightElementNode("img", "inline",
"single");
        image.CssClasses.Add("logo");
    }
}

```

```

        list.Children.Add(item1);
        list.Children.Add(item2);
        list.Children.Add(image);

        Console.WriteLine("--- Тестування LightHTML (Компонувальник) ---");
        Console.WriteLine($"Кількість дочірніх елементів:
{list.ChildrenCount}");
        Console.WriteLine("\n[Виведення InnerHTML]:");
        Console.WriteLine(list.InnerHTML);
        Console.WriteLine("\n[Виведення OuterHTML]:");
        Console.WriteLine(list.OuterHTML);

        Console.ReadKey();
    }
}

```

Результат:

```

--- Тестування LightHTML (Компонувальник) ---
Кількість дочірніх елементів: 3

[Виведення InnerHTML]:
<li class="menu-item">Головна</li><li class="menu-item">Про нас</li><img class="logo"/>

[Виведення OuterHTML]:
<ul class="menu-list"><li class="menu-item">Головна</li><li class="menu-item">Про нас</li><img class="logo"/></ul>

```

Рис. 5.1. Робота шаблону Компонувальник

Завдання 6: Легковаговик

1. За допомогою свого *LightHTML* з завдання 1 перетворіть текст книги в HTML верстку за такими правилами:

- а. Перший рядок має бути елементом <h1>
- б. Якщо в рядку менше 20 символів - це має бути елемент <h2>
- с. Якщо рядок починається з пробільного символу – це має бути <blockquote>.

```

ACT V
Scene I. Mantua. A Street.
Scene II. Friar Lawrence's Cell.
Scene III. A churchyard; in it a Monument belonging to the Capulets

Dramatis Personæ

ESCALUS, Prince of Verona.
MERCUTIO, kinsman to the Prince, and friend to Romeo.
PARIS, a young Nobleman, kinsman to the Prince.
Page to Paris.

```

d. В будь-якому іншому випадку – елемент `<p>`

2. Покажіть скільки займає все дерево Вашої верстки, коли воно повністю утримується в пам'яті процесу.

3. Використайте Легковаговик на Ваших класах HTML елементів, щоб зменшити споживання пам'яті.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

За допомогою *LightHTML* перетворено текст книги з файлу *book.txt* в HTML верстку за такими правилами: перший рядок є елементом `<h1>`, рядок менше 20 символів – `<h2>`, рядок із пробільного символу – `<blockquote>`, а в будь-якому іншому випадку – елемент `<p>`. У програмі показано, скільки займає все дерево верстки, коли воно повністю утримується в пам'яті процесу. Для зменшення споживання пам'яті на класах HTML елементів використано Легковаговик.

Лістинг Program.cs:

```

namespace Flyweight
{
    public abstract class LightNode
    {
        public abstract string OuterHTML { get; }
    }

    public class LightTextNode : LightNode
    {
        public string Text { get; }
        public LightTextNode(string text) => Text = text;
        public override string OuterHTML => Text;
    }
}

```

```

public class LightElementNode : LightNode
{
    public string TagName { get; }
    public string DisplayType { get; }
    public string ClosureType { get; }
    public List<LightNode> Children { get; } = new List<LightNode>();

    public LightElementNode(string tagName, string displayType, string
closureType)
    {
        TagName = tagName;
        DisplayType = displayType;
        ClosureType = closureType;
    }

    public override string OuterHTML
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in Children) sb.Append(child.OuterHTML);
            return $"<{TagName}>{sb}</{TagName}>";
        }
    }
}

public class FlyweightTag
{
    public string TagName { get; }
    public string DisplayType { get; }
    public string ClosureType { get; }

    public FlyweightTag(string tagName, string displayType, string
closureType)
    {
        TagName = tagName;
        DisplayType = displayType;
        ClosureType = closureType;
    }
}

public class FlyweightFactory
{
    private readonly Dictionary<string, FlyweightTag> _tags = new();

    public FlyweightTag GetTag(string tagName, string displayType, string
closureType)
    {
        if (!_tags.ContainsKey(tagName))
        {

```

```

        _tags[tagName] = new FlyweightTag(tagName, displayType,
closureType);
    }
    return _tags[tagName];
}
}

public class FlyweightElementNode : LightNode
{
    private readonly FlyweightTag _tag;
    public List<LightNode> Children { get; } = new List<LightNode>();

    public FlyweightElementNode(FlyweightTag tag)
    {
        _tag = tag;
    }

    public override string OuterHTML
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in Children) sb.Append(child.OuterHTML);
            return $"<{_tag.TagName}>{sb}</{_tag.TagName}>";
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;

        string filePath = "book.txt";
        if (!File.Exists(filePath))
        {
            Console.WriteLine("Помилка: Створіть файл book.txt у папці
проекту!");
            return;
        }

        string[] bookLines = File.ReadAllLines(filePath);

        Console.WriteLine($"--- Успішно завантажено весь текст книги:
{bookLines.Length} рядків ---");
        Console.WriteLine("\n--- Демонстрація верстки перших 5 рядків за
правилами ---");
        for (int i = 0; i < Math.Min(5, bookLines.Length); i++)
        {

```

```

        string line = bookLines[i];
        string tag = "p";
        if (i == 0) tag = "h1";
        else if (line.StartsWith(" ") || line.StartsWith("\t")) tag =
"blockquote";
        else if (line.Length < 20 && !string.IsNullOrEmpty(line))
tag = "h2";

        Console.WriteLine($"<{tag}> {line.Trim()} </{tag}>");
    }

    int multipliers = 10;

    GC.Collect();
    GC.WaitForPendingFinalizers();
    long memBeforeHeavy = GC.GetTotalMemory(true);

    List<LightNode> heavyTree = new List<LightNode>();
    for (int i = 0; i < multipliers; i++)
    {
        for (int j = 0; j < bookLines.Length; j++)
        {
            string line = bookLines[j];
            LightElementNode node;

            if (j == 0 && i == 0) node = new LightElementNode("h1",
"block", "paired");
            else if (line.StartsWith(" ") || line.StartsWith("\t"))
node = new LightElementNode("blockquote", "block", "paired");
            else if (line.Length < 20 &&
!string.IsNullOrEmpty(line)) node = new LightElementNode("h2", "block",
"paired");
            else node = new LightElementNode("p", "block", "paired");

            node.Children.Add(new LightTextNode(line));
            heavyTree.Add(node);
        }
    }
    long memAfterHeavy = GC.GetTotalMemory(true) - memBeforeHeavy;

    heavyTree = null;
    GC.Collect();
    GC.WaitForPendingFinalizers();

    long memBeforeFlyweight = GC.GetTotalMemory(true);
    FlyweightFactory factory = new FlyweightFactory();
    List<LightNode> flyweightTree = new List<LightNode>();

    for (int i = 0; i < multipliers; i++)
    {

```

```

        for (int j = 0; j < bookLines.Length; j++)
        {
            string line = bookLines[j];
            FlyweightTag tag;

            if (j == 0 && i == 0) tag = factory.GetTag("h1", "block",
"paired");
            else if (line.StartsWith(" ") || line.StartsWith("\t")) tag
= factory.GetTag("blockquote", "block", "paired");
            else if (line.Length < 20 &&
!string.IsNullOrEmpty(line)) tag = factory.GetTag("h2", "block",
"paired");
            else tag = factory.GetTag("p", "block", "paired");

            FlyweightElementNode node = new FlyweightElementNode(tag);
            node.Children.Add(new LightTextNode(line));
            flyweightTree.Add(node);
        }
    }
    long memAfterFlyweight = GC.GetTotalMemory(true) -
memBeforeFlyweight;

    Console.WriteLine("\n--- Фінальні результати аналізу пам'яті про-
цесу ---");
    Console.WriteLine($"Обсяг звичайної верстки всього тексту:
{memAfterHeavy / 1024.0:F2} KB");
    Console.WriteLine($"Обсяг верстки з Легковаговиком:
{memAfterFlyweight / 1024.0:F2} KB");
    Console.WriteLine($"Економія оперативної пам'яті: {((memAfterHeavy
- memAfterFlyweight) / (double)memAfterHeavy) * 100:F1}%");

    Console.ReadKey();
}
}
}

```

Результат:

```

--- Успішно завантажено весь текст книги: 5650 рядків ---

--- Демонстрація верстки перших 5 рядків за правилами ---
<h1> The Project Gutenberg eBook of Romeo and Juliet </h1>
<blockquote> </blockquote>
<p> This eBook is for the use of anyone anywhere in the United States and </p>
<p> most other parts of the world at no cost and with almost no restrictions </p>
<p> whatsoever. You may copy it, give it away or re-use it under the terms </p>

--- Фінальні результати аналізу пам'яті процесу ---
обсяг звичайної верстки всього тексту: 9326,80 KB
обсяг верстки з Легковаговиком: 8446,15 KB
Економія оперативної пам'яті: 9,4%

```

Рис. 6.1. Результат роботи шаблону Легковаговик

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				21
Змн.	Арк.	№ докум.	Підпис	Дата		

Висновок: під час виконання лабораторної роботи було досягнуто поставлену мету та отримано практичні навички з реалізації структурних шаблонів проектування. У ході виконання завдань було успішно розроблено та протестовано програмні рішення з використанням шаблонів Адаптер, Декоратор, Міст, Компонувальник, Проксі та Легковаговик.

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.3	Арк.
		Левківський В. Л.				22
Змн.	Арк.	№ докум.	Підпис	Дата		